Student Management System

Submitted By

Student Name	Student ID
Toukir Ahmed	251-15-216
Sadek Ahmed Raj	251-15-511
Tanvir Hossain	251-15-203
Khatheja Islam Mysha	251-15-031

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE123: Data Structure in the Computer Science and Engineering Department**

DAFFODIL INTERNATIONAL UNIVERSITY

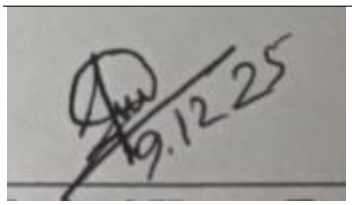
Dhaka, Bangladesh

December 12, 2025

DECLARATION

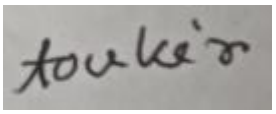
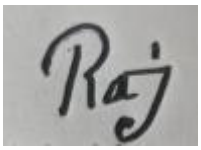
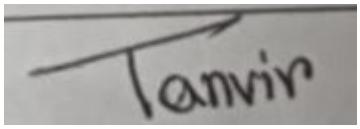
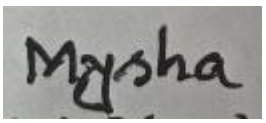
We hereby declare that this lab project has been done by us under the supervision of **Raja Tariqul Hasan Tusher**, **Assistant Professor**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:



Raja Tariqul Hasan Tusher
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Submitted by

 Toukir Ahmed ID: 251-15-216 Dept. of CSE, DIU	 Sadek Ahmed Raj ID: 251-15-511 Dept. of CSE, DIU
 Tanvir Hossain ID: 251-15-203 Dept. of CSE, DIU	 Khatheja Islam Mysha ID: 251-15-031 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Learning Outcome (CO)

CO's	Statements
CO1	**Design the concept of fundamentals of data structure for solving real-life problem having complex engineering attributes.**
CO2	**Solve a real-life problem having application of abstract data type created within the scope of complex engineering problem solving with the apply modern tools.**
CO3	**Apply Data Structure to solve real world problems member in diverse team.**

Table 2: Mapping of CO, PO, Blooms, KP and CEP

COs	CO Statements	POs	Learning Domains	Knowledge Profile	Complex Engineering Problem	Complex Engineering Activities
CO1	Design the concept of fundamentals of data structure for solving real-life problem having complex engineering attributes.	PO3	PO1,PO2,C3	K5	EP1	N/A
CO2	Solve a real-life problem having application of abstract data type created within the scope of complex engineering problem solving by applying modern tools with the involvement of Stockholders.	PO5	P2, C3	K6	EP2, EP6	N/A
CO3	Apply Data Structure to solve real world problems member in diverse team.	PO9	P3, A4	N/A	N/A	N/A

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Chapter 1

Introduction

This chapter provides an overview of the Student Management System and outlines the purpose, background, and importance of developing such a system. It also introduces the core features, scope, and structure of the project that will be discussed in the following chapters.

1.1 Introduction

Managing student information is a critical component of any academic institution. A Student Management System (SMS) centralizes all student-related data into a structured format, allowing efficient storage, retrieval, updating, and monitoring of student information. This project develops an SMS using the C programming language, focusing on linked list structures for dynamic data handling and file operations for permanent data storage.

1.2 Motivation

The primary motivation behind developing this project is to gain practical mastery over core data structures, which include linked lists, dynamic memory allocation, modular programming, file management, and algorithmic design. Creating a real-world executable system enhances conceptual understanding and prepares developers for larger software projects.

1.3 Objectives

The objectives of the project include:
- Designing a functional student management system using C.
- Demonstrating real-life application of linked lists.
- Ensuring secure login through password-based authentication.
- Implementing persistent storage through binary files.
- Allowing admins to manage all student records.
- Allowing students to access and update limited profile information.

1.4 Feasibility Study

The project is technically feasible because C provides all the necessary features such as pointer manipulation, file handling, modular programming, and data abstraction. Economically, the project requires no external infrastructure. Operational feasibility is supported by its simple text-based interface, enabling ease of use for both admin and student roles.

1.5 Gap Analysis

Although many student management solutions exist, most rely heavily on GUI-based frameworks or database systems. There is a lack of lightweight, low-level implementations that demonstrate how such a system can be built entirely from scratch. This project fills that gap by creating a robust SMS using linked lists and binary file storage without depending on external libraries.

1.6 Project Outcome

The system successfully enables administrators to create, update, search, delete, and store student

information. Students can log in to update their personal details. The overall outcome demonstrates practical applications of data structures.

Chapter 2

Proposed Methodology/Architecture

This chapter describes the architecture and methodology of the Student Management System. It explains the system analysis, design specification, overview, and tmethodology used to manage student information efficiently.

2.1 Requirement Analysis & Design Specification

The system requires two user roles: Admin and Student. Admin users can perform CRUD operations (Create, Read, Update, Delete) on student records. Students can log in and update their profile information. The requirements include storing the following data fields:

- Student ID
- Name
- Department
- Address
- Phone Number
- Gender
- Blood Group
- Date of Birth
- CGPA
- Password for authentication

2.1.1 System Overview

The system uses a singly linked list as the core data structure, where each node represents a student record. File I/O operations ensure data persistence even after closing the program. The program is divided into multiple modules for maintainability, such as login module, admin module, file module, and data handling module.

2.1.2 Proposed Methodology/ System Design

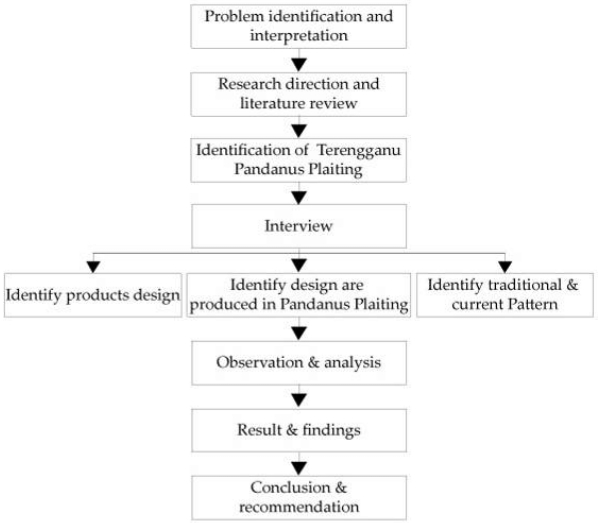


Figure 2.1: This is a sample diagram

2.1.3 UI Design

The system uses a text-based interface with clear instructions and structured menus for different roles. Although simple, this design ensures clarity, ease of use, and accessibility. The hierarchical menu structure guides users through admin or student tasks efficiently.

2.2 Overall Project Plan

The project followed the structured SDLC steps:
1. Requirement Analysis
2. System Design
3. Implementation
4. Testing & Debugging
5. Documentation & Reporting

Chapter 3

Implementation and Results

This chapter presents the implementation details of the proposed system implementation, its performance, and discusses the results obtained from various experiments.

3.1 Implementation

The program is implemented in C using modular functions. All student data is stored in a structure. Linked list operations such as insertion, searching, deletion, and updating were implemented manually. Binary files are used for storing data permanently. The admin module includes full access and modification capabilities, while student access is restricted for security reasons.

3.2 Performance Analysis

The linked list structure ensures that insertion and deletion operations are efficient (O(1) for insert, O(n) for search). Because the system stores data in RAM using dynamic allocation, it is fast and lightweight. File operations introduce minimal overhead.

3.3 Results and Discussion

The system successfully processes all essential student management functionalities. Testing shows that CRUD operations work accurately. Admin and student authentication behave as expected, and the binary file storage system reliably stores and restores student data upon program restart.

Here are some screenshots for proper output :-

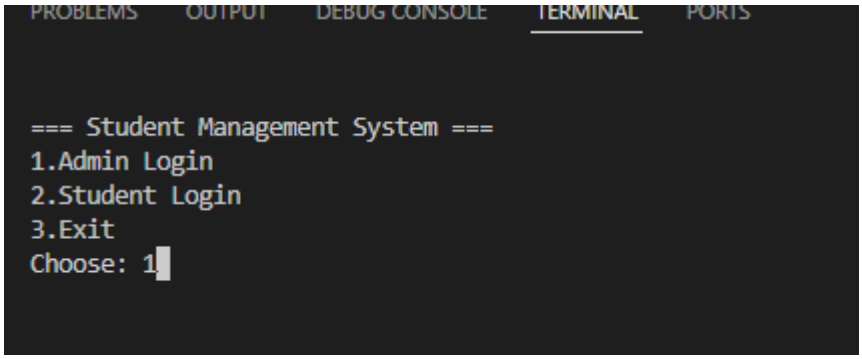


Figure 3.1 : main entry menu

Figure Description : This screen shows the **main entry menu** of the Student Management System. It displays three options: Admin Login, Student Login, and Exit. The user is expected to type a number to choose an option. The current input shown is “1”, meaning the user selected the Admin Login path. The interface is entirely text-based, indicating that the system is running in a console environment without any graphical components.

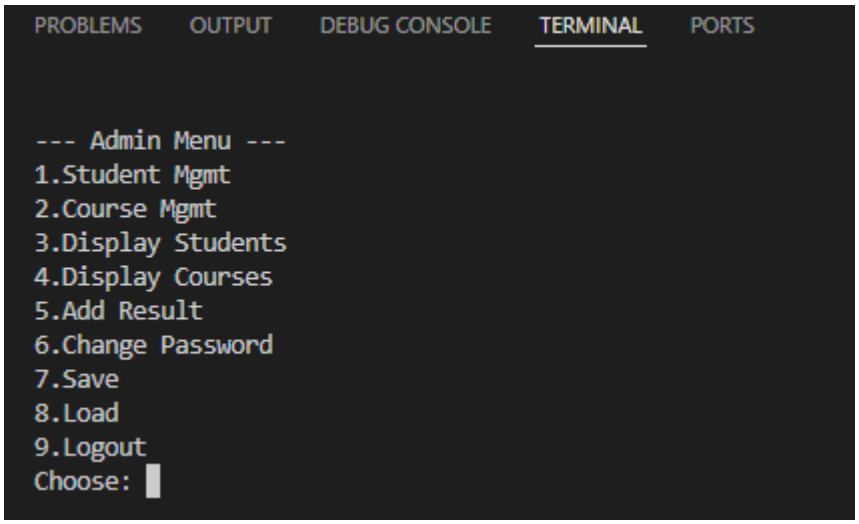


Figure 3.2 : Main menu

Figure Description : This figure shows the **Admin Menu**, which appears after a successful admin login. It lists all major system functions the admin can control. The options cover student management, course management, displaying records, adding results, updating the password, saving/loading data, and logging out. The interface is entirely text-based, and the admin must enter a number to select a specific operation.

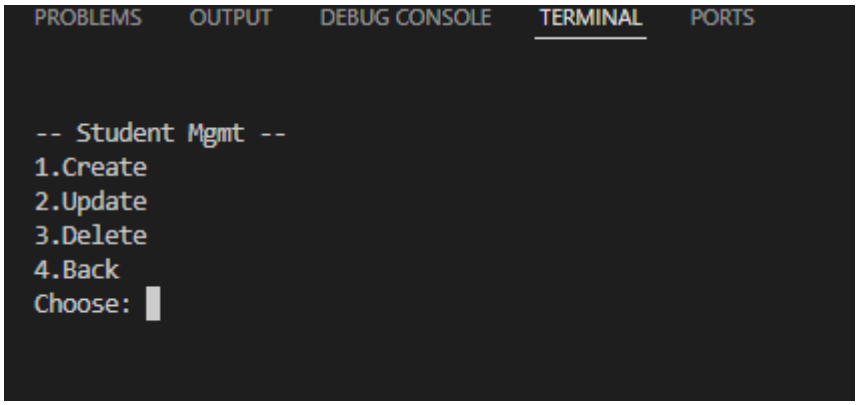


Figure 3.3 : Student management options

Figure Description : This screen shows the Student Management submenu within a console-based system. It presents four options: Create, Update, Delete, and Back. The interface header reads "-- Student Mgmt --". At the bottom, a "Choose:" prompt is displayed, indicating that the system is waiting for the user to enter a number to select an option. The entire interface is text-based, showing that the program is running in a terminal environment without graphical elements.

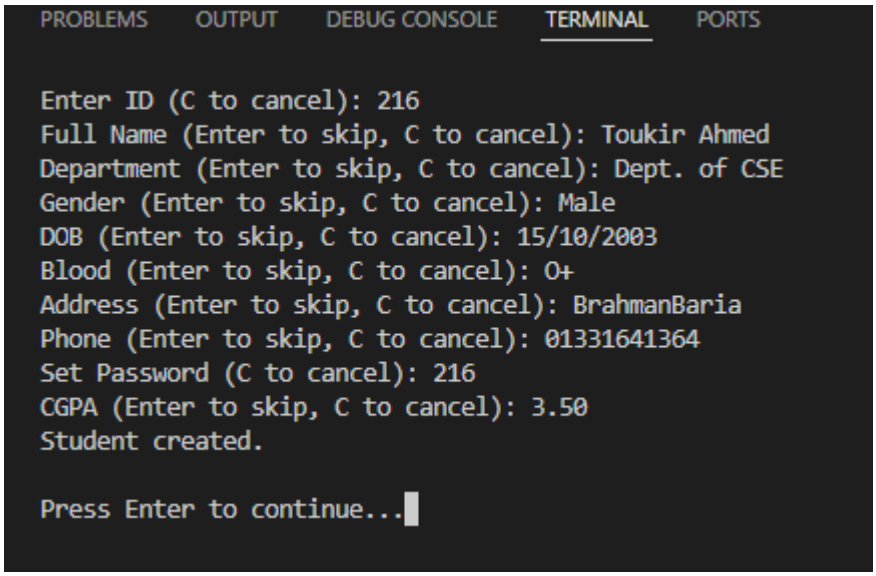


Figure 3.4 : New student profile creation

Figure Description : This screen shows the student profile creation form inside the console-based Student Management System. The system prompts the user to enter various details such as ID, Full Name, Department, Gender, Date of Birth, Blood Group, Address, Phone Number, Password, and CGPA. Each field allows skipping or canceling using specific keys. After all inputs are provided, the terminal displays the confirmation message "Student created." followed by a prompt instructing the user to press Enter to continue. The interface is entirely text-based, indicating a command-line environment.

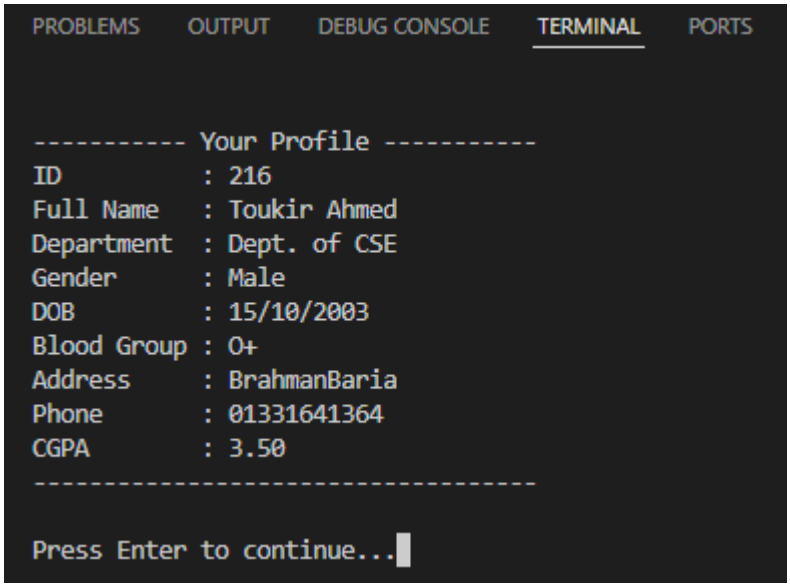


Figure 3.5 : Created new student profile

Figure Description : This screen shows the profile display section of the console-based Student Management System. It presents the student's stored information, including ID, Full Name, Department,

Gender, Date of Birth, Blood Group, Address, Phone, and CGPA. The data is shown in a neatly formatted text layout under the heading “Your Profile.” At the bottom, the system prompts the user to press Enter to continue. The interface is fully text-based, indicating that it is running in a terminal environment.

1. Update student details :

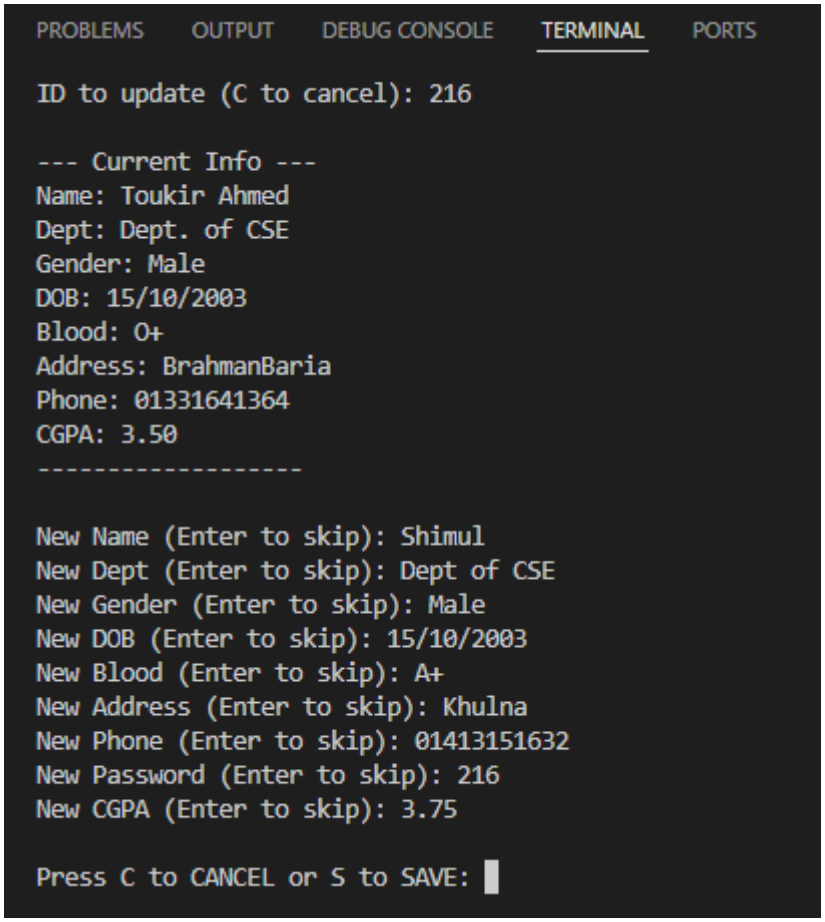


Figure 3.6 : Updating student details

Figure Description : This screen displays the student update interface of the Student Management System. It shows the currently stored information of a selected student, including their name, department, gender, date of birth, blood group, address, phone number, and CGPA. Below the current details, the system prompts the user to enter new values for each field, allowing updates while also letting the user skip any field by pressing Enter. After entering the new information, the interface asks the user to press **C** to cancel or **S** to save, indicating that the system is fully text-based and runs inside a console environment.

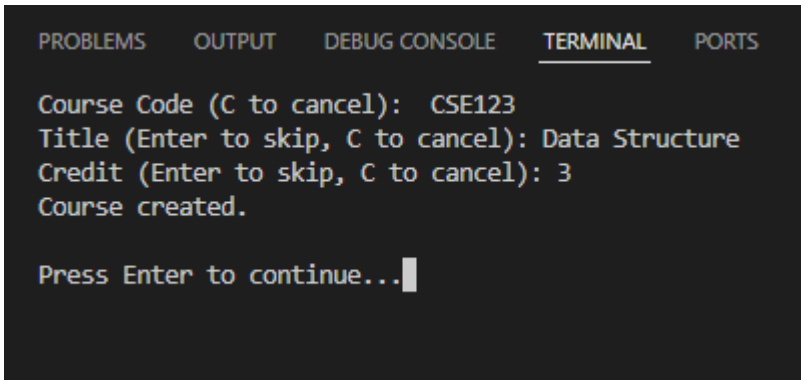


Figure 3.7 : Creating course for students

Figure Description : This screen shows the course creation interface of the Student Management System. The user is prompted to enter a course code, a course title, and the course credit value, with options to skip fields by pressing Enter or cancel by typing **C**. After the required information is provided, the system confirms that the course has been created. The interface is fully text-based, indicating that the system operates in a console environment and waits for the user to press Enter to continue.

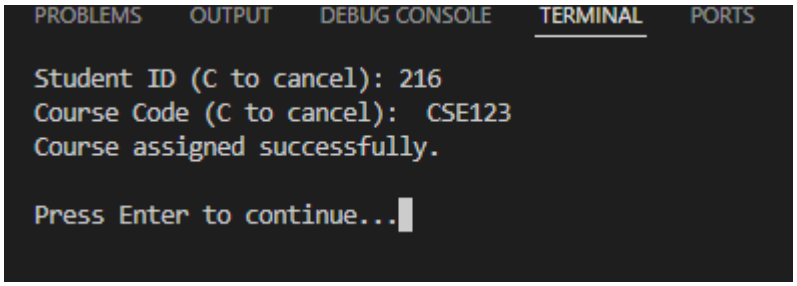


Figure 3.8 : Assigning course to students

Figure Description : This screen shows the course assignment interface of the Student Management System. The system prompts the user to enter a student ID and a course code, with the option to cancel by typing **C**. After the inputs are provided, the system confirms that the course has been assigned successfully. The interface is fully text-based, indicating that the program is running in a console environment and waits for the user to press Enter to continue.

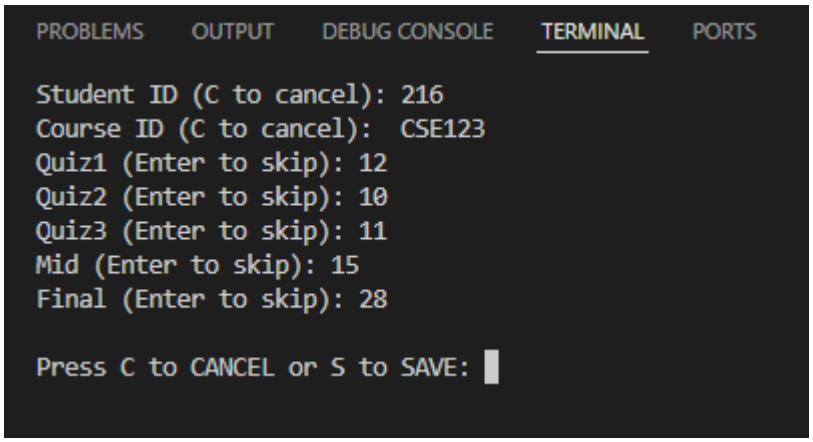


Figure 3.9 : Adding results

Figure Description : This screen displays the grade entry interface for a student within the Student Management System. It prompts the user to input a student ID and course ID, followed by the marks for multiple assessments such as Quiz1, Quiz2, Quiz3, Mid, and Final. Each field allows skipping by pressing Enter or canceling by typing **C**. After entering all required marks, the system waits for the user to press **S** to save or **C** to cancel. The entire interface is text-based, showing that the system is running in a console environment.

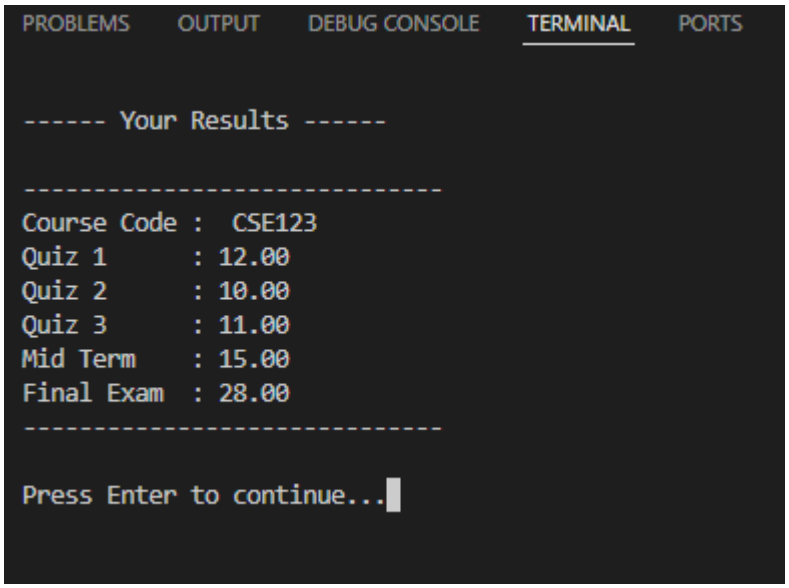


Figure 3.10 : Result-viewing interface

Figure Description : This screen shows the result-viewing interface from the student login section of the Student Management System. It displays the student's assessment results for a specific course, including scores for Quiz 1, Quiz 2, Quiz 3, the Mid Term, and the Final Exam. The layout is fully text-based, formatted with dividers for clarity, and the system waits for the user to press Enter to continue, indicating that it operates in a simple console environment.

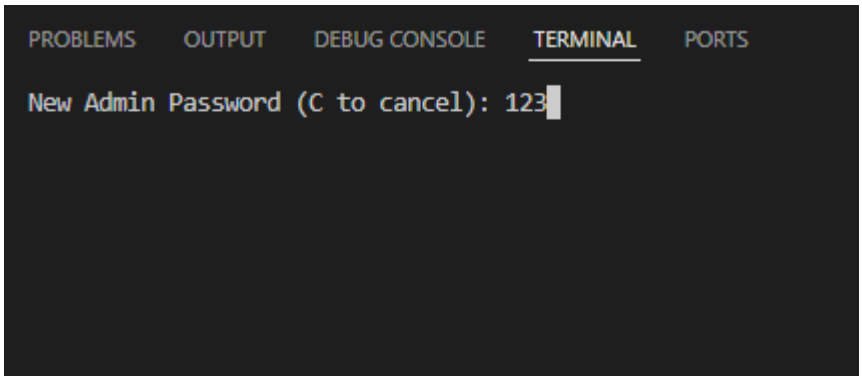


Figure 3.11 : Password-changing interface

Figure Description : This screen shows the password-changing interface for the admin user in the Student Management System. The console prompts the user to enter a new admin password, with an option to press **C** to cancel the operation. The current input displayed is **"123"**, indicating that the user is typing the new password. The interface is completely text-based, showing that the system is operating in a terminal environment without any graphical elements.

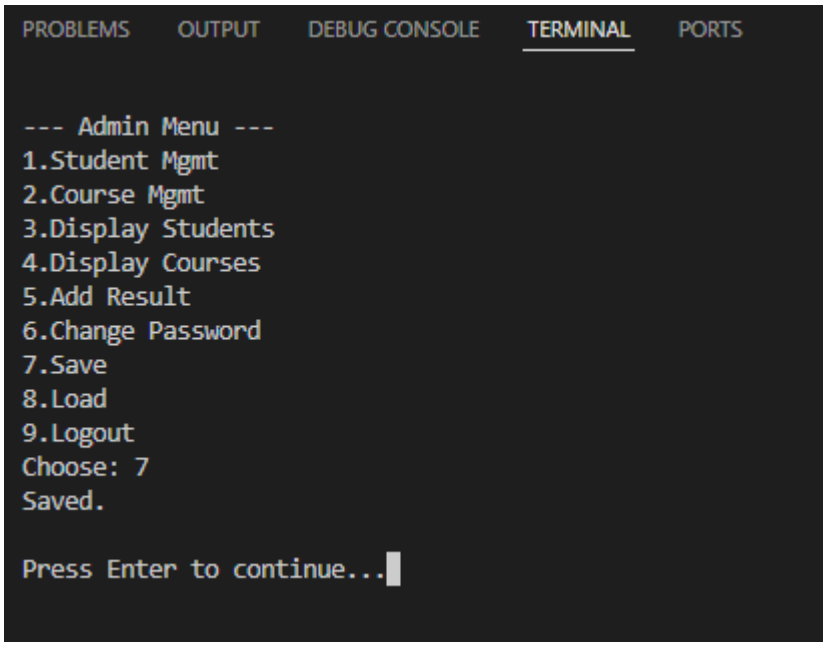


Figure 3.12 : Saving all

Figure Description : This figure shows the "Save" option in the Admin Menu of the Student Management System. When the administrator selects option 7, the system saves all current records, including student information, course details, and results. A confirmation message "Saved." is displayed, indicating that the data has been successfully stored. The system then prompts the user to press Enter to continue.

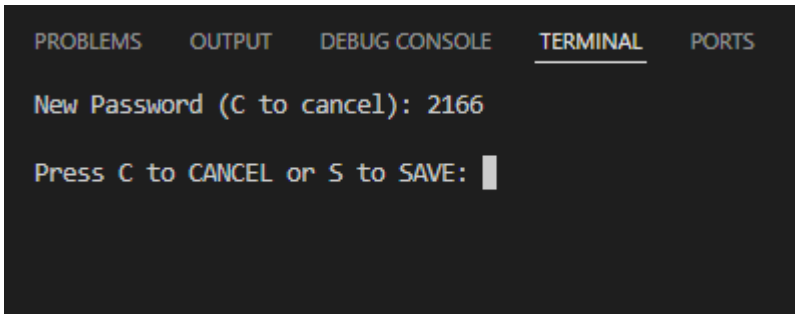


Figure 3.13 : Password-changing interface

Figure Description : This screen shows the password-changing interface for a student account in the Student Management System. The console prompts the user to enter a new password, with an option to press **C** to cancel. The input displayed is **"2166"**, indicating the new password typed by the user. After entering the password, the system asks the user to choose whether to cancel or save the change by pressing **C** or **S**. The interface is entirely text-based, showing that the system is running in a simple terminal environment without graphical components.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the engineering standards followed in the development of the Student Management System and evaluates its societal, environmental, ethical, and sustainability impacts. It also covers project management practices, cost analysis, and the mapping of complex engineering problems with program outcomes.

4.1 Impact on Society, Environment and Sustainability

The project promotes digital transformation within academic institutions by reducing paperwork. Digitization enhances record accuracy, speeds up workflow, and ensures long-term sustainability.

4.1.1 Impact on Life

- "The system minimizes manual workload for students, teachers, and administrative officers."
- "It reduces errors in academic record-keeping, improving fairness and transparency in student evaluation."
- "Fast access to information improves students' academic planning."

4.1.2 Impact on Society & Environment

- "Digitization reduces paper usage significantly, decreasing environmental waste."
- "Parents and institutions get faster, accurate information about academic progress."
- "Institutions become more efficient, reducing administrative bottlenecks."

4.1.3 Ethical Aspects

The project ensures ethical handling of student data through password-based authentication. Confidential information is stored securely and accessed only by authorized individuals.

4.1.4 Sustainability Plan

The system can be enhanced by integrating MySQL or cloud-based databases. Additionally, turning it into a web application would increase accessibility and scalability.

4.2 Project Management and Team Work

The development of the Student Management System followed a structured project management approach to ensure timely and efficient completion. The project tasks were divided among team members based on their strengths, including UI design, backend development, database structuring, documentation, and testing. Weekly meetings were conducted to review progress, identify obstacles, and reassign tasks when necessary. A simple timeline and workflow plan were maintained to track deliverables, which helped reduce delays and maintain consistency across all components. Team members collaborated through shared tools, communicated regularly

to resolve technical issues, and supported each other during integration and testing phases. A clear distribution of responsibilities and continuous coordination improved productivity, minimized errors, and resulted in a stable and functional system. Additionally, a basic cost analysis was included to understand resource usage, and an alternate budget plan was prepared to reflect real-world implementation scenarios. Overall, effective teamwork and disciplined project management ensured smooth execution of the entire system development cycle.

Table : Financial Analysis

S.N.	Component	Estimated Cost (BDT)
1	Software Tools	300
2	Hardware Usage	500
3	Transportation	600
4	Printing & Photocopy	300
5	Documentation & Binding	500
6	UI/UX Resources	800
	Total Estimated Cost	3,000

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO's	**Justification**
PO3	Able to design solutions and develop system components or processes to address complex engineering problems, such as creating data structures and system logic for real-life applications.
PO5	Able to apply modern engineering and IT tools—like software, programming environments, and data-processing tools—to analyze, design, and solve complex problems involving abstract data types and real-life scenarios.
PO9	Able to function effectively both independently and as a member of a diverse team in solving real-world problems, including the development and application of data structures.

4.3.2 Complex Problem Solving

In this section, provide a mapping with problem solving categories. For each mapping add subsections to put rationale (Use Table 4.2). For P1, you need to put another mapping with

Knowledge profile and rational thereof.

Table 4.2: Targeted Attributes for DS-Complex Engineering Problem

EP1	EP2	**EP6**
Dept of Knowledge required - Requires advanced understanding of data structures beyond textbook examples; involves algorithm design, optimization, and data abstraction.	Range of Conflicting Requirements - Must deal with multiple conflicting goals such as time vs. space efficiency, performance vs. complexity, or accuracy vs. computation cost.	**Extent of Stakeholder Involvement and Level of Conflicting Requirements** – Solution should reflect real-world application contexts such as transportation systems, networks, logistics, or scheduling — incorporating stakeholders considerations.

Chapter 5

Conclusion

This chapter summarizes the overall development, outcomes, and limitations of the Student Management System project. It also highlights the key findings and the practical value of the work completed.

5.1 Summary

The Student Management System successfully demonstrates the use of fundamental data structures and file handling in C. The project is a practical implementation of theoretical knowledge gained in the course.

5.2 Limitation

Although the system works, it has several limitations. It does not include a graphical user interface, which reduces usability. There is no database support, so all data is stored in files, making the system less scalable and harder to maintain. It also lacks multi-user access and cannot operate in an online environment. These limitations restrict the system to very small-scale use.

5.3 Future Work

Several improvements can significantly enhance the system. Adding database integration would provide better storage, security, and scalability. A GUI-based interface would make the system more user-friendly. Introducing role-based access control (Admin, Teacher, Student) would allow multi-user functionality. Exporting reports, automating result generation, and integrating cloud or web support could turn the project into a complete academic management platform. A future extension could also include mobile app development for easier access.

References

[1] Jon Kleinberg and Eva Tardos. *Algorithm design*. Pearson Education India, 2006.
[2] **GeeksforGeeks — Student Management System in C**
https://www.geeksforgeeks.org/student-management-system-in-c/
Accessed: 08 Dec 2025
[3] Brian W. Kernighan & Dennis M. Ritchie. The C Programming Language. PHI, 1988.
[4] **TutorialsPoint — File Handling in C**
https://www.tutorialspoint.com/cprogramming/c_file_io.htm
Accessed: 06 Dec 2025
[6] **StudyTonight — Data Structures in C**
https://www.studytonight.com/data-structures/
Accessed: 04 Dec 2025
[7] "Software Engineering: A Practitioner's Approach" – Roger S. Pressman, McGraw-Hill

**Link of this project source code :
https://drive.google.com/file/d/1Igv6SKKjkoC92f6Z77q0uNGzD5-WS_RC/view?usp=sharing

APPENDIX

Daffodil International University
Daffodil Smart City (DSC), Birulia, Savar, Dhaka-1216, Bangladesh

Date: 9 December 2025
To
The Deputy Director
Daffodil International University
Daffodil Smart City (DSC), Birulia, Savar, Dhaka-1216

Subject: Request for Permission to Collect Academic Data for "Student Management System" Project

Dear Sir/Madam,
We the undersigned students of the Department of Computer Science & Engineering, are currently working on a course project titled **"Student Management System."**
To complete the project accurately, we need access to **basic non-confidential academic records**, organizational workflow and general operational structure related to student management.
To ensure our system design matches real academic processes, we seek your permission to collect limited information from the department office.

Purpose of Data Collection

1. **Student Information Structure** – Understanding the required fields such as ID, name, program, semester, courses etc.
2. **Course & Enrollment Workflow** – Observing how course registration and student enrollment are managed.
3. **Result Management** – Understanding how marks, grading, and semester results are stored/processed
4. **Administrative Workflow** Collecting non-confidential details about daily student-related operations

All information will be used **strictly for academic purposes only** and no confidential or sensitive data will be requested or stored.

Proposed Date & Time

15.12.25

Reference :
Raja Tariqul Hasan Tusher
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

We request your kind approval and an official signature to proceed with the data collection

Sincerely,

1. Toukir Ahmed — ID: 251-15-216
2. Sadek Ahmed Raj ID: 251-15-511
3. Tanvir Hossain — ID 251-15-203
4. Khatheja Islam Mysha ID: 251-15-031

Signature:

15/12/2025